Micro-Architecture of LW Driven Bubble-Free Five-Stage Pipelined RISC-V Processor Core for Energy Constraint Low-End Application

Sujeet Kumar and Kailash Chandra Ray, *Member, IEEE*

***Abstract*—RISC-V has gained attention in recent times for its open-source ISA, modularization, and extensible features that facilitate customization in various edge computing applications. These RISC-V-based modern processors often use a pipeline architecture and frequently load data from memory to enhance the throughput. However, bubbles due to load-word (Data hazard) hazards will inevitably arise in the pipeline, causing degradation in CPI. The out-of-order pipeline processor can handle the bubble through scheduling but it introduces WAR (write after read) and WAW (write after write) making architecture more complex and not suitable for low-power applications. Hence, this paper proposes a novel lw-driven bubble-free five-stage pipelined RV32I processor core based on open-source RISC-V ISA, i.e. the proposed core is an in-order RISC-V processor core that can handle bubbles without any clock penalty due to stall. The core is implemented using Verilog HDL and prototyped on the commercially available Xilinx Artix-7 FPGA with a resource requirement of 1271 LUTs and 641 Registers. A notable 19.37% improvement in CPI is achieved compared to the state-of-the-art stalling algorithm on the same RV32I processor core. Comparing the proposed design to the traditional stall method, the score improved significantly by 14%, achieving a core mark score of 1.06 $CoreMark/MHz$. The dynamic power consumption of the core is 0.9 mW/MHz and is best suited for energy-constrained applications.**

***Index Terms*—RISC-V ISA, pipeline processor, load word hazard, data hazard control, bubble.**

I. Introduction

THE evolution of technology has made the application domain more flexible in terms of customization. Energy-efficient application-specific cores are designed to perform specific tasks more efficiently than general-purpose cores. These cores are optimised for specific application or workloads, such as Internet of Things (IoT), fault-tolerant, or edge computing. By focusing on a specific task, energy-efficient application-specific cores can reduce the number of transistors and clock cycles required to perform the task, resulting in lower power consumption. Those are often used in Edge computing and IoT devices where power constraints are a major concern. These cores can also be used to offload specific tasks from general-purpose cores, allowing them to remain in a low-power state for longer periods of time. Nowadays, RISC V ISA [1] is becoming an attractive research domain to design and develop energy-efficient domain-specific processor architecture. Due to the energy-constrained environment, many custom designs are employed to meet the energy requirements of such offshore applications. To further improve the performance, pipelining is introduced in the RISC-V processor cores. Based on RISC-V ISA, various cores are created or under development for the specific application. Rocket [2], [3] is the first RISC-V micro-architecture by the same researchers that created the instruction set. RV64G ISA-based 6-stage single-issue in-order pipeline processor. ETH Zurich created Pulpino [4], [5], [6], an open-source microcontroller system based on RV32IMC RISC-V core. Clusters of RISC-V CPUs share closely connected data memory on this architecture. IIT Madras researchers are also developing a resilient processor for radiation-prone areas, including nuclear and space applications, using RISC-V based 5stage pipeline Shakti-F [7] processor core. Different Shakti variants including E, C, and I, are being developed [8] having 3, 6 and deep pipelined RISC-V stages, respectively. Some more RISC-V cores include ORCA [9], mriscv [10], VexRiscv [11], LowRISC [12], and RI5CY [13]. The pipeline architectures confront data hazards which degrades the performance of the processor core. Data forwarding techniques reduce pipeline processor data hazards, still load word (LW) hazards remain unresolved. These LW data hazards lead to bubble generation in the output of the pipelined processors.

Limitations and Challenges: The RISC-V pipelined architecture methods presented in [14] have expanded the idea of data hazard and its handling through forwarding. The LW hazards will cause the program to crash. As a result, resolving data hazards is a crucial prerequisite for ensuring the system's normal behaviour. Generally every program may encounter an LW hazard. The usual technique adopted to avoid an LW hazard is a one-cycle stall pipeline causing an additional bubble. In [15], a software-based method is presented to mitigate this hazard by reordering the instructions, although this method only avoids the condition and does not address it. The software technique to resolving data hazards is less difficult, but it has a greater impact on processor throughput than the hardware approach [16]. When this happens, stalling and bubbles are unavoidable. Therefore, it motivates

Received 8 September 2024; revised 21 January 2025 and 5 June 2025; accepted 3 November 2025. Date of publication 11 November 2025; date of current version 12 January 2026. This work was supported by the C2S Program of Ministry of Electronics and Information Technology, Government of India, New Delhi under Grant EE-9/2/2021-R&D-E. This article was recommended by Associate Editor X. Zeng. *(Corresponding author: Sujeet Kumar.)*

The authors are with the Department of Electrical Engineering, Indian Institute of Technology Patna, Patna, Bihar 801106, India (e-mail: sujeet_2021ee08@iitp.ac.in; kcr@iitp.ac.in).

Digital Object Identifier 10.1109/TCSI.2025.3629838

1549-8328 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on August 24,2026 at 02:50:22 UTC from IEEE Xplore. Restrictions apply.

TABLE I
TEST SAMPLE PROGRAM

Instructions	Assembly Code	Machine Code
Instruction 1	addi R3, R0, 4	00400193H
Instruction 2	addi R6,R0, 2	00200313H
Instruction 3	sw R3, 0(R6)	00332023H
Instruction 4	add R3, R3, R8	003181b3H
Instruction 5	lw R3, 0(R6)	00032183H
Instruction 6	addi R3, R3, 8	00818193H
Instruction 7	add R3, R9, R8	003181b3H

for mitigation of bubble in the RISC-V based pipeline soft cores implemented on Xilinx FPGA [17], [18], [19], [20], and [21].

Contribution: In order to mitigate the LW-induced stall and bubble, a novel micro-architecture of five-stage pipelined RV32I processor has been proposed. The core is implemented utilizing 32-bit RISC-V ISA [1] base extension 'I' to make it energy-efficient. It just includes the 36 RV32I ISA instructions that are essential. This paper's contributions include the development of an architecture and bubble mitigation algorithm that is integrated into the design of a five-stage pipelined RISC-V processor core that is lw-driven bubble-free.

The organization of this paper is as follows, Section II briefly discusses data hazards and related works to handle it. Section III presents a novel algorithm and micro-architecture of lw-driven bubble-free five-stage RISC-v pipelined processor core. Section IV discusses the FPGA prototype and its evaluation. Finally, Section V concludes the paper.

II. RELATED WORK

A data hazard in a RISC-V-based pipelined processor occurs when there is a conflict or reliance between consecutive instructions in the pipeline, which can result in inaccurate results or pipeline stalls. There are three categories of data hazards Read-After-Write (RAW), Write-After-Read (WAR), and Write-After-Write (WAW).

Out of these three data hazards, in-order RISC-V-based pipeline architecture is only prone to RAW hazard.

When a data hazard occurs, the pipeline is forced to halt, so alternative techniques, such as forwarding, interlocking [22], dynamic scheduling [23], and asynchronous designs [24], have been utilised to alleviate the hazard. The following three techniques are commonly employed to avoid data hazards in pipelined RISC-V processors are Reordering [23], Forwarding or Data Bypassing [15], and Stall or Pipeline Bubble [22].

- Reordering: Out-of-order execution is employed on more complex pipelined processors. Instructions can run out of order if their data dependencies are met. It necessitates the presence of mechanisms in the processor that allow instructions to be reordered before being committed to the architectural state [23], [25]. In the given tableI *sw* $R3, 0(R6)$ instruction's R3 and R6 data are dependent upon previous instructions *addi* $R3, R0, 4$ and *addi* $R6, R0, 2$. In this case, a data hazard occurs if the instruction is reshuffled in a manner so that there is no data dependency between 3 consecutive instructions in the case of a five-stage pipeline architecture.

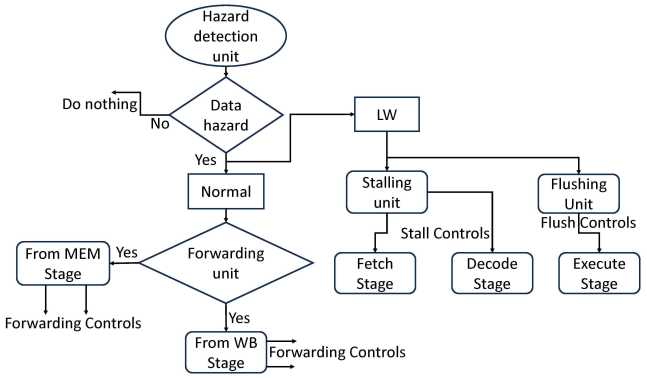


Fig. 1. Flow chart of data hazard handling.

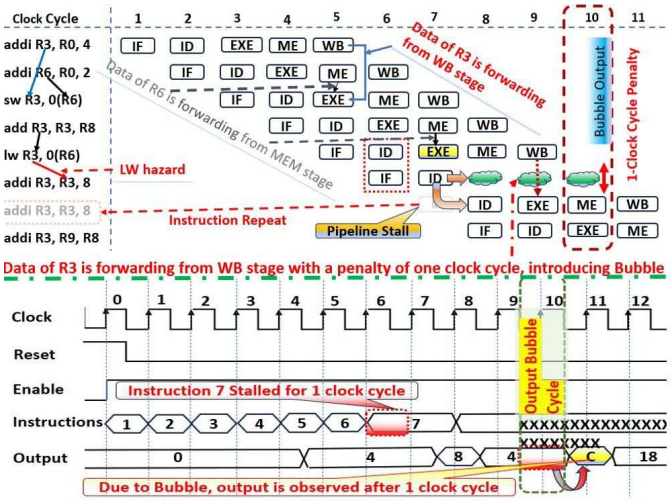


Fig. 2. LW hazard induced bubble in pipeline stages.

- Forwarding (Data Bypassing): Data can be transferred straight from the execution step of an instruction to a subsequent stage that requires the data using forwarding, eliminating the need to wait for the data to be written to the register file. It can assist in resolving RAW hazards without stalling. The state-of-the-art forwarding algorithm is shown in Fig. 1. when data hazard occurs between *sw* $R3, 0(R6)$, *addi* $R3, R0, 4$ and *addi* $R6, R0, 2$ instructions. Forwarding will solve the data hazards while forwarding the data from the Memory (MEM) stage or the writeback(WB) stage. The forwarding of data from these two stages is shown in Fig. 2.
- Stall (Pipeline Bubble): When a RAW hazard occurs, the processor can put a stall, or bubble, in the pipeline. This indicates that no additional instructions are fetched until the hazard has been handled, allowing the dependent instruction to finish [26]. The data dependence of R3 occurs in an example of an LW instruction such as *lw* $R3, 0(R6)$ and its subsequent instruction *addi* $R3, R3, 8$. Still, it cannot be forwarded because of a memory delay of 2 cycles, thus causing a LW hazard. This unique data hazard is where the introduction of a bubble will occur. The processor's performance will be affected because of the clock-cycle penalty. The stall algorithm is shown in

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on August 24,2026 at 02:50:22 UTC from IEEE Xplore. Restrictions apply.

Fig. 2, where the presence of a bubble is evident as a penalty.

Handling data hazards is vital to pipeline design since it affects the processor's performance and correctness. Pipelined processors frequently employ a combination of such approaches to mitigate the impact of data hazards while retaining efficient execution [27]. Data hazard handling is in application-oriented RISC-V-based pipelined architecture given the main priority. The conventional way to solve the data hazard is through the forwarding and stalling algorithm shown in Fig. 1. A new study employing an SSR approach to lower the LW hazards is reported in [28]; nevertheless, the technique is unclear and the study is not conventionally sound.

The other way to solve this LW bubble issue is to use an out-of-order processor [15]. The out-of-order processor multiple copies of hardware to resolve the issue of LW hazard but due to this it suffers from WAR (write after read) and WAW (write after write) data hazard [24]. To resolve this issue the architecture requires different hardware blocks which leads to greater power dissipation and area overhead. Due to its complex architecture and power requirement, the out-of-order processor is not a suitable choice for energy-constraint applications. Hence, the mitigation of bubbles in an in-order pipeline RISC-V architecture should be addressed.

A typical RISC-V program's instruction count that is put into the processor core's instruction memory is shown by the equation 1.

$$T_{Con.}(I) = I + \sum_{n=0}^{n} p_n(q_n + 2r_n) \tag{1}$$

where $T_{Con.}$ is the total number of instruction cycles to complete a general program, n is the number of loops, I is the number of instructions outside any loop p_n is the number of iterations of the nth loop, q_n is the number of instructions inside the nth loop without LW hazard and r_n is the number of instructions inside the nth loop having LW hazard. Additionally, we assume that the LW threat only manifests within loops.

III. Proposed Lw-Driven Bubble-Free Five-Stage Pipelined RISC-V Processor Core

The five-stage pipelined RISC-V processor core is designed and implemented having the support of 36 RISC-V ISA instructions. The LW hazard is the only data hazard that is not resolved by bypassing and needs to stall, which introduces a bubble at the output. Due to this special type of data hazard, pipeline architecture has to sacrifice one clock cycle. This causes a bubble to be generated at the execution stage, which then propagates through the pipeline stages at the output. A unique test program has been built to demonstrate all the forms of data hazards described in the Table I.

Instruction *addi* $R3, R3, 8$ has a unique data reliance on Instruction *lw* $R3, 0(R6)$ and Instruction *add* $R3, R3, R8$, although this may be handled by the hazard unit based on the memory stage data forwarding across the writeback stage. However, there is an issue with LW instruction; therefore, stalling enters into the picture to resolve this, resulting in bubble generation and one clock cycle penalty. Subsequent to this specific data hazard next instruction *addi*$R3, R3, 8$ and instruction *add*$R3, R9, R8$ share identical data reliance on their earlier stages, which is handled by the data hazard unit. The sole LW-induced data hazard has a clock cycle penalty that must be reduced using any specific method.

In Fig. 2, it can be observed that because of LW data hazard, there has been staling conducted on instruction *addi* $R3, R3, 8$; hence, due to this, a bubble was produced in that stage resulting penalty of 1 clock cycle. This issue can be resolved by incorporating a novel micro-architecture for bubble mitigation.

A. Proposed Bubble Mitigation Algorithm

Before designing the proposed micro-architecture, an algorithm to mitigate bubbles is presented here. The Proposed algorithm to mitigate bubble due to stalling in the case of LW hazard is depicted in Fig. 3. The bubble fixing control unit manages the LW bubble mitigation and detection. Case1, Case2, and Case3 are the three categories for LW hazards shown in the Fig. 3; Case2 is extremely rare, while Case3 is the rarest. The cases of LW hazard are given below-

- Case1: It occurs as a result of data hazard caused by LW. Data cannot be sent from the memory stage to the execute stage if a register depends on the previous adjacent load word instruction. This will cause the architecture to stall and create a bubble. For example *lw* $R3, 0(R6)$ and *addi* $R3, R3, 8$. where a single Lw data hazard is detected.
- Case2: Case2 occurs because of two successive data hazards. The proposed approach is unable to produce the right address if the value of the LW instruction register (R6) depends on its un-updated value. The solution to this problem is to forward the updated value of the register (R6) from the memory stage one cycle before generating the right address, as the proposed approach advances the address one clock earlier. For example, *addi* $R6, R0, 4$, *addi* $R5, R2, R1$, *lw* $R3, 0(R6)$, *addi* $R3, R3, 8$.
- Case3: Case3 occurs because of the architecture's hardware limitations. The adjacent prior instruction should be a non-LW instruction since the Proposed algorithm advances the address one clock ahead of time; otherwise, hardware conflicts may occur. For example,*lw* $R4, 0(R5)$ *lw* $R3, 0(R5)$ *addi* $R3, R3, 8$ or if forwarding from the memory stage is acknowledged, hardware limitations prevent it from being completed one stage earlier like *addi* $R6, R0, 4$ *lw* $R3, 0(R6)$ *addi* $R3, R3, 8$.

Case1 occurs as a result of a data hazard caused by LW. Data cannot be sent from the memory stage to the execute stage if a register depends on the previous adjacent load word instruction. This will cause the architecture to stall and create a bubble.

The bubble fixing control units' control signals (DM_ReadF, ALU_OUTF, $DATA_LW$) are initialised to zero at the start of the process. The LW data hazard and regular data hazard are distinguished by a hazard detection unit. The forward unit is activated to fix the issue by forwarding the data from memory or write back stage to execute stage in the event of a typical data hazard. The

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on August 24,2026 at 02:50:22 UTC from IEEE Xplore. Restrictions apply.

Bubble Fixing Control Unit handles the data hazard in the event of a LW hazard.

Case3 necessitates stalling but can be eliminated by compiler re-ordering; the proposed algorithm can only alleviate cases 1 and 2.

The algorithm first detects LW hazards in the decode-execute stage and then generates the address of data memory ($Addr.DM$) along with mux control signals (DM_ReadF, ALU_OUTF, $DATA_LW$). Further, the mux signals along with the data memory address are propagated through the pipeline register into the execute stage. The mux controls signal allows the address of data memory ($Addr.DM$) to pass into the memory stage while keeping all the signals and data in the execute stage. As two back-to-back consecutive LW hazard detections is rare so the algorithm disables the generation of address and mux control signals (DM_ReadF, ALU_OUTF, $DATA_LW$) in the next cycle so the normal process is restored.

After the completion of the execution stage, data is available and accessible in the memory stage to be transmitted back, if required, in the next instruction.

The approach is built such that there is no substantial influence on the clock latency of the Pipeline processor.

Equation 1 provides a generalized equation of the total cycle count of a general program used in RISC-V processors, which may be used for empirical algorithm analysis. The revised equation of the total cycle count, which may be defined as equation 2, is represented as $Tpro.$ after the integration of the proposed bubble mitigation architecture.

$$T_{pro.}(I) = I + \sum_{n=0}^{n} p_n(q_n + r_n) \tag{2}$$

When developing the equation, it is assumed that the LW hazard only arises within loops. Therefore, equation 3 may be used to describe the overall number of cycle improvements.

$$Total\ Instruction\ saving = T_{Con.}(I) - T_{pro.}(I) \tag{3}$$

Equation 5 illustrates how the equation is further simplified by equation 4.

$$= I + \sum_{n=0}^{n} p_n(q_n + 2r_n) - \left(I + \sum_{n=0}^{n} p_n(q_n + r_n) \right) \tag{4}$$

$$= \sum_{n=0}^{n} p_n r_n. \tag{5}$$

The proposed approach is used to enhance the clock cycle, and the result is represented by the following expression.

$$Cycle\ Improvement = \frac{\sum_{n=0}^{n} p_n r_n}{I + \sum_{n=0}^{n} p_n(q_n + 2r_n)} \tag{6}$$

B. Proposed Lw-Driven Bubble-Free Micro Architecture

Based on the bubble mitigation algorithm presented in Fig. 3, the architecture is designed and shown in Fig. 5. A 5-bit two comparator examines the data dependence of the LW instruction on the preceding signals along with WD_SRC_EXE signal is used to determine LW signal. The

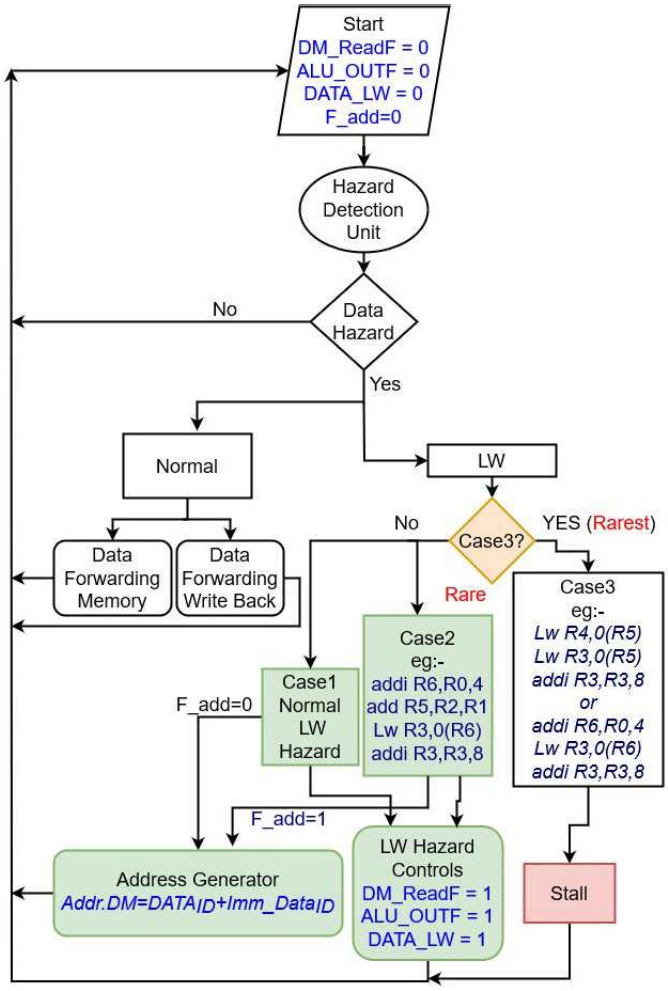


Fig. 3. Proposed bubble mitigation algorithm in RISC-V based pipeline architecture.

architecture used to generate $MUX_CONTROL_SIGNALS$ (signals which controls the forwarding muxs) in the case of LW data hazard (Bubble detection). The proposed lw-driven bubble-free architecture integrates a five-stage RV32I-based processor with a bubble mitigation control unit and is shown in Fig. 4. To ensure that the latency of the architecture is not impacted, prior to propagating through the EXE stage, the address generation unit first computes the address of the LW instruction in the Decode stage. When the LW hazards is detected during the execute stage, the address generation unit transfers the data memory address (already calculated in decode stage) to the Memory stage one cycle earlier. In case1 and case2 of LW hazard, the mux control signal of the LW bubble fix control unit is set to 0 and 1, respectively. The address is, therefore, accessible to the data memory through the LW hazard unit during the execute stage when the LW instruction is run to compute the address of Data memory to operate. Since the data is accessible, bubble mitigation is achieved by having the data fed back from data memory to the ALU through the forwarding unit in the event of a LW hazard.

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on August 24,2026 at 02:50:22 UTC from IEEE Xplore. Restrictions apply.

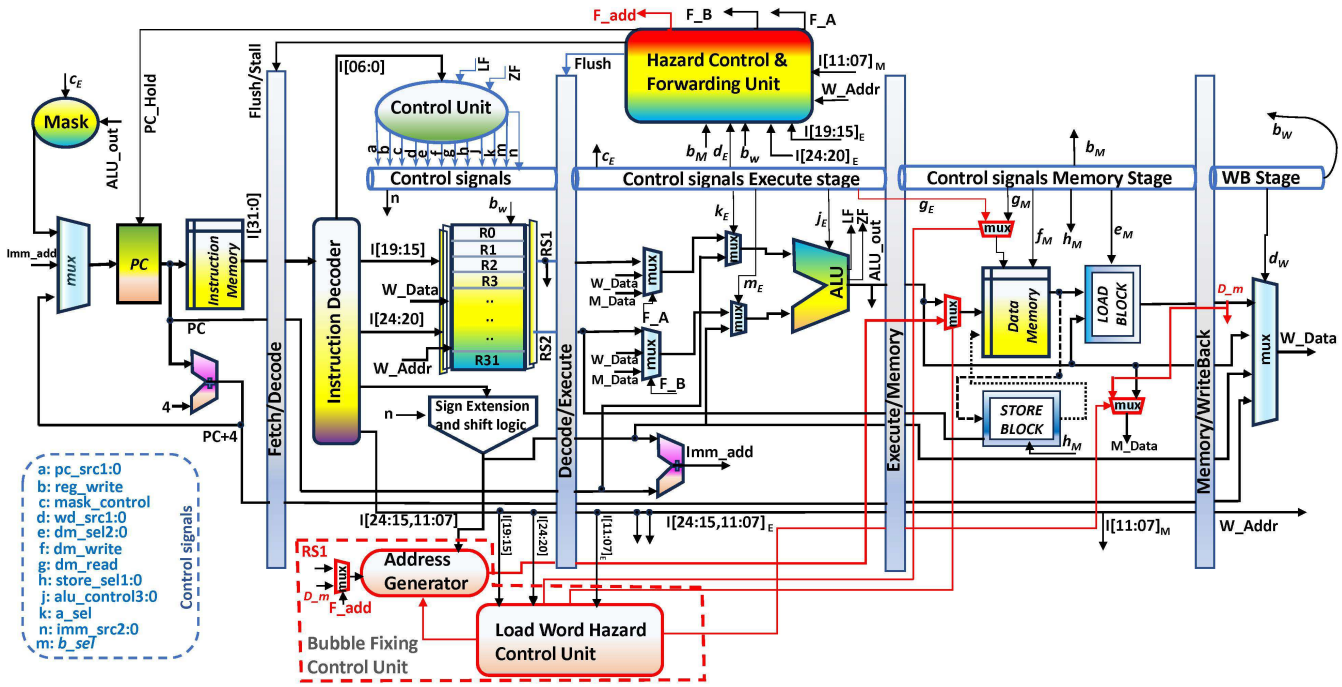


Fig. 4. Proposed lw-driven bubble-free five-stage pipelined RISC-V processor core.

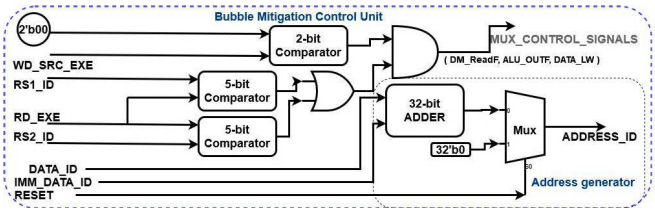


Fig. 5. Proposed bubble mitigation control unit architecture.

IV. HDL Simulation, FPGA Prototype and Evaluation

The proposed architecture of a lw-driven bubble-free five-stage pipelined RISC-V processor core is shown in Fig. 4 implemented using Verilog HDL (Hardware Description Language) and prototyped on commercially available FPGA chip and board in laboratory experimental setup.

A. HDL Simulation

The HDL simulation and functional verification of the proposed micro-architecture is carried out using the XILINX VIVADO EDA tool. The proposed LW-induced bubble mitigation control unit with a five-stage RV32I processor is implemented using Verilog HDL (Hardware Description Language).

To mitigate the LW-induced bubble issue, the proposed algorithm implemented in the five-stage RV32I pipeline architecture and HDL simulation of the proposed architecture is carried out. To address the many forms of data hazards and their mitigation by the proposed architecture, as well as to validate functionality, after loading the test program displayed in the Table I into the instruction memory, HDL simulation is run. The proposed design's HDL simulation of LW hazard stalling algorithm is presented in Fig. 6. As a simulation result, it is evident that the novel architecture can handle all types of data hazards without any stalling or flushing operation. In the simulated waveform, it is clearly observed that the expected output (0000000c) appears just after the previous value (0000 0004) without any clock delay. New instructions are acquired at each rising edge of the clock, and writing on the register bank file is done at the negative edge.

B. FPGA Prototype

The proposed architecture and the base architecture (without bubble mitigation) is prototyped on a commercially available Nexys4 DDR board with an Artix7 FPGA chip with the part number $XC7A100T-1CSG324C$. Logical synthesis is used to assess the area, latency, and power performance of the proposed bubble mitigation technique compared to the traditional stalling approach. The findings are shown in the Table II. As a result of synthesis, the critical path of the proposed lw-driven bubble-free architecture and coventional stalling architecture are expressed as the equation 7 and equation 8, respectively. The first-order timing complexity analysis on the critical path of the five-stage RV32I processor core with the proposed algorithm is carried out the results are expressed as-

$$T_{proposed} = T_{ID} + T_{rfr} + T_{mx2BF} + T_{AGBF}. \quad (7)$$

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on August 24,2026 at 02:50:22 UTC from IEEE Xplore. Restrictions apply.

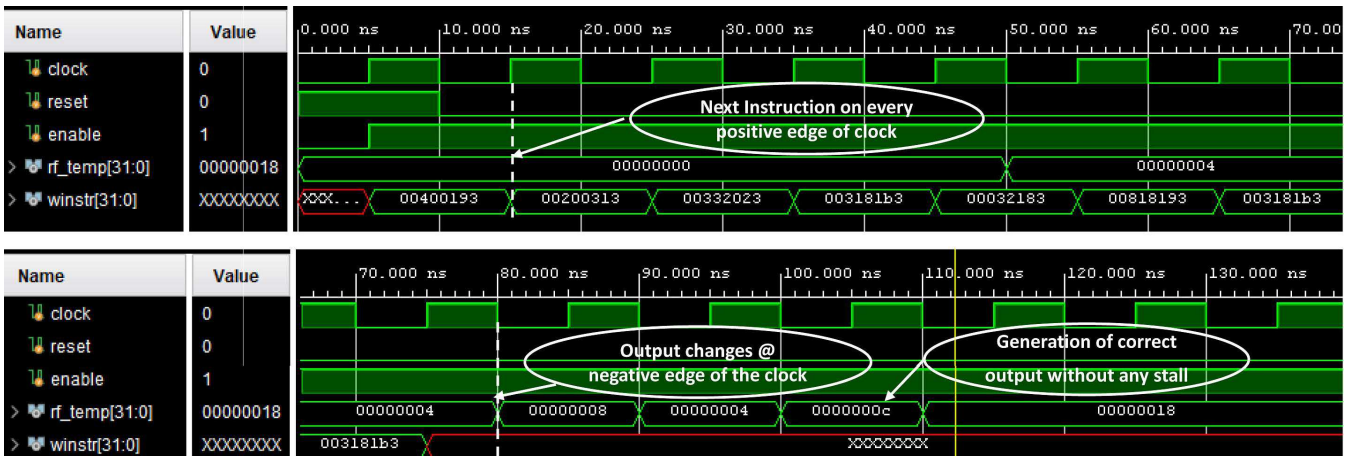


Fig. 6. Simulated waveform of LW hazard-induced bubble mitigation in pipeline stages using VIVADO EDA Tool.

TABLE II
PERFORMANCE COMPARISON

Parameter	RV32I[a]	**Proposed RV32I[b]**	Improvement
LUTs+Register	1716	1912	-11.42%
Max Delay (ns)	7.684	7.931	-3.2%
Max Frequency (MHz)	130	126	-3.0%
Power (mW)	121	**114**	**+5.8%**
mW/MHz	0.93	**0.90**	**+3.2%**

[a] RV32I processor core with state-of-art stalling technique
[b] RV32I processor core with proposed bubble mitigation technique

where T_{ID}, T_{rfr}, T_{mx2BF}, T_{AGBF} signifies the implemented delay of the instruction decode, register bank read, multiplexer of bubble fix unit and address generator of bubble fix unit, respectively. The proposed in-house five-stage pipelined architecture's critical path uses a traditional stalling method to manage the hazard of LW data after the first-order timing complexity analysis has described the critical path as

$$T_{Conventional} = T_{mux4WB} + T_{mux3ex} + T_{ALU} + T_{MASK} + T_{mux3IF} + T_{PC}. \quad (8)$$

where T_{mux4WB}, T_{mux3ex}, T_{ALU}, T_{MASK}, T_{mux3IF}, T_{PC} signifies the implemented delay of the mux of write back stage, mux of execute stage, ALU, MASK, Mux of fetch stage and program counter, respectively.

The critical path of the conventional stalling algorithm and proposed bubble mitigation algorithm of the Five-stage RV32I processor core are 7.684 ns and 7.931 ns, respectively. The calculation of the delay path is given in Table II, and the new critical path has a delay overhead of 0.3 ns only. Due to the new critical path being moved to the decode stage, the MEM stage was designed to function in parallel with the EXE stage for additional optimisation, resulting in a four-stage architecture. Because of this optimisation, the architecture will encounter a couple of serious issues with different types of LW and SW data hazards, which will cause additional bubbles to form. In order to eliminate these pipeline bubbles, this optimisation is reversed. The integration of the LW hazard control unit resulted in an overhead of +11.42% in the area and a negligible 3% latency. The total latency of the proposed

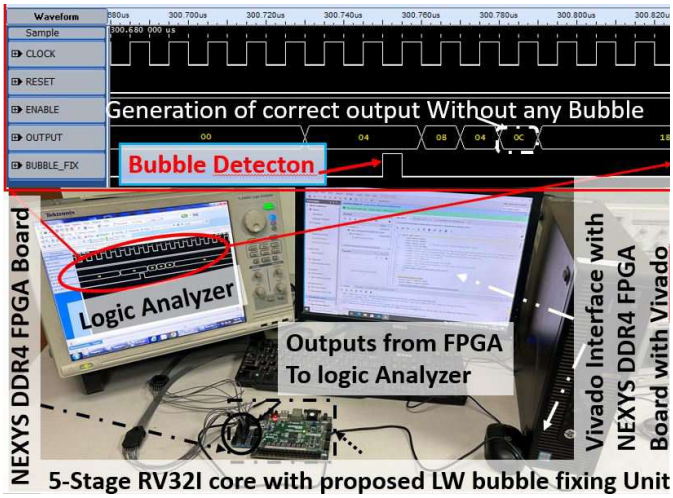


Fig. 7. Laboratory test setup of the proposed lw-driven bubble-free five-stage RISC-V micro-architecture.

core will not be much affected due to the increased latency of the decode stage. According to Table II, the proposed design has improved energy efficiency at the expense of a very slight latency increase.

The laboratory experimental setup for chip-level validation of the proposed micro-architecture using Tektronix TLA6402 Logic Analyser is shown in Fig. 7.

As shown in Fig. 7, when the LW bubble is detected by the LW bubble mitigation unit in the ID/EXE stage, it generates the bubble correction signal ($BUBBLE_FIX$). $BUBBLE_FIX$ signal passes the advance address generated in the ID stage to the MEM stage one clock cycle before. The experimental waveform of the test program shown in Table I running with 100 iterations is shown in Fig. 7. In the case of LW hazard, the proposed architecture achieves the expected output ($0000000C$) without any penalty. Additionally, Functional verification has been done using a lightweight bench-marking algorithm i.e., Fibonacci series generation [29], Correlation, quick sort algorithm [30], merge sort algorithm, Dijkstra, and results are reported in Table IV.

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on August 24,2026 at 02:50:22 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMPARISON OF PROPOSED BUBBLE MITIGATION TECHNIQUE WITH CONVENTIONAL STALLING ALGORITHM FOR RISC V-BASED PIPELINE PROCESSOR CORES

Experiment parameter		Ex-1	Ex-2	Ex-3	Ex-4	Ex-5	Ex-6	Ex-7	Ex-8	Ex-9	Ex-10	Average
LW-Type	Case1	1	2	3	4	5	6	7	8	9	10	5.5
	Case2	0	1	1	2	2	3	3	4	4	5	2.5
	Case3	0	0	0	0	1	1	1	1	1	2	0.7
Loop	Iterration1	10	10	20	50	100	100	200	150	200	250	109
	Iteration2	10	20	40	75	50	50	100	200	250	300	109.5
	Iteration3	10	50	50	100	20	50	150	100	300	500	133
Total Clock cycle Required	Stall Algorithm[a]	642	2335	3696	9328	8518	11234	25867	29458	48962	68765	20880.5
	Compiler Re-ordering[a]	642	2255	3586	8878	8178	10734	24517	27658	45762	64565	19677.5
	Bubble Mitigation[b]	**612**	**2095**	**3256**	**7978**	**7328**	**9434**	**21367**	**24058**	**39212**	**53015**	**18473.33**
	Compiler Re-ordering[b]	**612**	**2095**	**3256**	**7978**	**7158**	**9234**	**20917**	**23608**	**38462**	**50915**	**16423.5**
Total Cycle Saved	Normal	30	240	440	1350	1190	1800	4500	5400	9750	15750	4045
	Compiler Re-Ordering	30	240	440	1350	1360	2000	4950	5800	10500	17850	4452
Clock cycle Improvement(%)	**Normal**	**4.67**	**10.28**	**11.91**	**14.47**	**13.97**	**16.02**	**17.39**	**18.33**	**19.91**	**22.91**	**19.37**
	Compiler Re-Ordering	**4.67**	**10.28**	**11.91**	**14.47**	**15.97**	**17.81**	**19.14**	**19.69**	**21.45**	**25.96**	**16.14**

[a] RV32I processor core with state-of-art stalling technique
[b] RV32I processor core with proposed bubble mitigation technique

TABLE IV
COMPARISON WITH LOW POWER STATE OF ART CORE

Cores	SCR1 [17]	PicoRV [21]	CV32e40p [18]	E21 [19]	N22 [20]	Proposed RV32I[a]	**Proposed RV32I[b]**
Slice LUTs	4402	917	6515	18486	6919	1123	1271
Slice Registers	2327	583	2127	3885	2984	593	641
DSP	1	1	4	2	2	0	**0**
Average Dynamic Power (mW/MHz)	42	16	1.37	1.67	2.12	1.04	**0.90**
Pipeline Stages	4	2	5	3	2	5	5
LW Hazard Handling Technique	Stall	Stall	Stall	Stall	Stall	Stall	**Bubble Mitigation**
CPI	1.42	1.37	1.33	1.35	1.34	1.36	**1.21**

[a] RV32I processor core with state-of-art stalling technique
[b] RV32I processor core with proposed bubble mitigation technique

As a result of the experimental setup the proposed core with bubble mitigation control unit is able to achieve 114 *mW* of power at a maximum frequency of 126 *MHz*.

C. Evaluation and Comparison

The impact of the proposed method over the traditional stalling algorithm has been demonstrated using the modified program with various iterations and examples of LW potential hazards. After putting the program into the FPGA, 10 tests have been prepared and carried out as post-synthesis of the proposed core. In addition to the proposed core, every state-of-the-art core has been loaded with the same programs on an FPGA prototype on the same ARTIX 7 board. Table III displays the specifics of the experiments as well as their outcomes. Since traditional stalling occurs in all state-of-the-art low-power cores, the proposed bubble mitigation approach has been able to improve 19.37% of total clock cycles. Fig. 8 presents the findings from all of the experiments along with the proposed bubble mitigation along with conventional stalling algorithms. Further compiler reordering is applied to both the algorithm and data are reported in Table III. When comparing the compiler reordering to the stall technique, an average of 1203 clock cycles saving is reported. The proposed bubble mitigation algorithm, on the other hand, saves 4045 clock cycles in total. Compiler reordering significantly enhances clock cycle saving, because the proposed design only stalled in the LW case3 type. Applying the compiler reorder to the

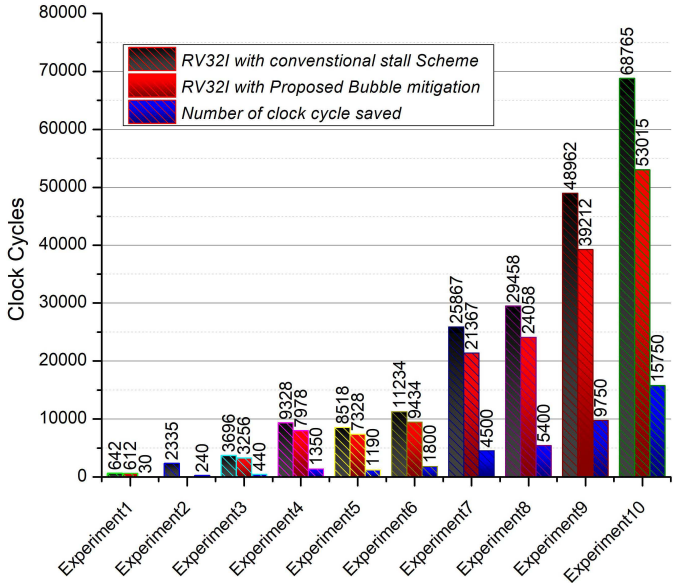


Fig. 8. Experimental results of 10 experiments to show the efficacy of the proposed bubble mitigation algorithm over stalling algorithm.

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on August 24,2026 at 02:50:22 UTC from IEEE Xplore. Restrictions apply.

proposed bubble mitigation approach has been shown to save an average of 4452 clock cycles, which will increase efficiency even further. Compiler reordering, for instance, will increase clock saving by 1050 clock cycles in experimentation 10. The clock cycle improvement increased from 22.91% to 24.43% in this particular case.

In order to evaluate the efficacy of the proposed processor core which includes either a state-of-the-art stalling unit or bubble mitigation control unit, the results of RV32I processor core with state-of-the-art stalling method and proposed bubble mitigation method are presented in Table IV along with other state-of-the-art similar energy-efficient processor cores. With respect to energy constraint, the following benchmark methods were assessed in light of the features covered in [31].

- Fibonacci: This rudimentary program with basic instructions was used for preliminary testing and is known as the Fibonacci series generation.
- Dijkstra: In a network, this method determines the shortest path between any two nodes.
- Correlation: A common arithmetic and jump instruction used in signal processing is correlation. Thus, appropriate for assessing low-end energy constraint edge nodes.
- Quicksort: An unsorted array of 50 items with random integers is the input. After sorting the input array, the sorted input array is output. It is therefore preferable for analysing the data hazard analysis since it requires several memory load instructions.
- Mergesort: An unsorted array of 50 items with random integers is the input. After sorting the input array, the sorted input array is output. It is therefore preferable for analysing the data hazard analysis since it requires several memory load instructions.

$$CPI = \frac{CPU\ Time}{Total\ instructions * Clock\ cycle\ time} \quad (9)$$

The CPI of all the soft cores are evaluated using equation 9 and reported in the Table IV. Here, it is observed that the performance in terms of CPI (Cycle per Instruction) improves 10% compared to the RV32I processor core with a state-of-the-art stalling technique. The Power measurements of FPGA are tool-based measurements. After the implementation of the logic over FPGA, the test vector information is provided to the power measurement tool, which will measure the specific program's power [32]. Energy efficiency is the primary measure in the design of the processor core for lightweight, energy-constrained edge computing devices. Therefore, power/frequency (mW/MHz) measurements are used to quantify core performance for these energy constraints [33], [34]. The performance in terms of dynamic power consumption of the proposed bubble-free RV32I processor core is compared with state-of-the-art similar range energy-efficient processor core as reported in Table IV. It is observed that dynamic power consumption of proposed lw-driven bubble-free RV32I processor core improves by 97.8%, 94.4%, 34.3%, 46.2% and 53.3% compared to energy-efficient processor core reported in [17], [18], [19], [20], and [21], respectively. Additionally, CPI of proposed lw-driven bubble-free RV32I processor core improves by 17.35%, 13.22%, 10%, 11.57% and 10.74% compared to energy-efficient processor core reported in [17], [18], [19], [20], and [21], respectively.

TABLE V
PERFORMANCE ANALYSIS USING COREMARK BENCHMARK

Processor Core	CoreMark (CoreMark/MHz)
TangNano9K [36]	1.04
PicoRV32 [21]	0.4
SCR1 [17]	1.01
Ibex (Micro-riscy) [37]	0.904
RV32EC [38]	0.58
NEORV32(small) [39]	0.26
MicroBlaze [40]	0.48
Paper [41]	0.64
Proposed RV32I[a]	0.93
Proposed RV32I[b]	**1.06**

[a] RV32I processor core with state-of-art stalling technique
[b] RV32I processor core with proposed bubble mitigation technique

It is evident that the proposed lw-driven bubble-free RV32I processor core can handle the LW bubble hazard without incurring any clock cycle penalty and can save a total of 19.37% clocks as per the experimental results shown in Table III.

Table V reports the results of running the CoreMark benchmark on the proposed softcore using the stall and bubble migration algorithm in order to further assess the effectiveness of the proposed algorithm. The CoreMark score of the low-end RISC-V-based softcore is also shown in Table V. When compared to the conventional stall algorithm, the proposed bubble mitigation algorithm achieves a 14% improvement in CoreMark score.

V. CONCLUSION

In this paper, a novel lw-driven bubble-free five-stage pipeline in-order RISC-V processor core is proposed to mitigate the bubble due to LW data hazard. The proposed lw-driven bubble-free pipelined processor core is designed using Verilog HDL and prototyped on a commercially available XILINX Artix-7 FPGA chip in a laboratory experimental setup. In the experiments, a notable 19.37% improvement in CPI is achieved by proposed bubble mitigation technique compared to the state-of-the-art stalling algorithm on the same RV32I processor core. A significant improvement in dynamic power consumption is also achieved comparing to the state-of-the-art similar energy-efficient processor core. When compared to the conventional stall algorithm, the proposed bubble mitigation algorithm achieves a 14% improvement in CoreMark score. Preventing bubbles without imposing clock penalties is the primary objective of the proposed lw-driven bubble-free RV32I processor core, which can also lead to a significant increase in energy efficiency. Therefore, the proposed lw-driven bubble-free RV32I processor core mitigates the LW hazard without causing a clock cycle penalty and achieves energy efficiency compared to a similar range state-of-the-art processor core and is best suitable for low-end energy constraint applications.

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on August 24,2026 at 02:50:22 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] K. Asanović and D. A. Patterson, "Instruction sets should be free: The case for RISC-V," Dept. EECS, Univ. California, Oakland, CA, USA, Tech. Rep. UCB/EECS-2014-146, 2014.

[2] B. Zimmer et al., "A RISC-V vector processor with simultaneous-switching switched-capacitor DC–DC converters in 28 nm FDSOI," *IEEE J. Solid-State Circuits*, vol. 51, no. 4, pp. 930–942, Apr. 2016.

[3] Y. Lee et al., "A 45nm 1.3 GHz 16.7 double-precision GFLOPS/W RISC-V processor with vector accelerators," in *Proc. 40th Eur. Solid State Circuits Conf. (ESSCIRC)*, Sep. 2014, pp. 199–202.

[4] M. Gautschi, M. Schaffner, F. K. Gürkaynak, and L. Benini, "An extended shared logarithmic unit for nonlinear function kernel acceleration in a 65-nm CMOS multicore cluster," *IEEE J. Solid-State Circuits*, vol. 52, no. 1, pp. 98–112, Jan. 2017.

[5] F. Conti et al., "An IoT endpoint system-on-chip for secure and energy-efficient near-sensor analytics," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 64, no. 9, pp. 2481–2494, Sep. 2017.

[6] M. Gautschi et al., "Near-threshold RISC-V core with DSP extensions for scalable IoT endpoint devices," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 25, no. 10, pp. 2700–2713, Oct. 2017.

[7] S. Gupta, N. Gala, G. S. Madhusudan, and V. Kamakoti, "SHAKTI-F: A fault tolerant microprocessor architecture," in *Proc. IEEE 24th Asian Test Symp. (ATS)*, Nov. 2015, pp. 163–168.

[8] P. Corner, P. George, A. Sahoo, A. Menon, and V. Kamakoti, "1.1 the shakti processor program," Version 1.1, Indian Inst. Technol. Madras, Chennai, Tamil Nadu, Tech. Rep., 2018, vol. 2, no. 3. [Online]. Available: http://52.172.171.174/show.article.php?id=64

[9] *Orca Rv32im*. Accessed: Jul. 21, 2024. [Online]. Available: https://github.com/vectorblox/orca

[10] *Mriscv*. Accessed: Jul. 21, 2024. [Online]. Available: https://github.com/onchipuis/mriscv

[11] A. E. Wilson, S. Larsen, C. Wilson, C. Thurlow, and M. Wirthlin, "Neutron radiation testing of a TMR VexRiscv soft processor on SRAM-based FPGAs," *IEEE Trans. Nucl. Sci.*, vol. 68, no. 5, pp. 1054–1060, May 2021.

[12] Z. Mohseni and P. Reviriego, "Reliability characterization and activity analysis of lowRISC internal modules against single event upsets using fault injection and RTL simulation," *Microprocessors Microsystems*, vol. 71, Nov. 2019, Art. no. 102871.

[13] *Ri5cy A Small 4-Stage Risc-V Core*. Accessed: Jul. 21, 2024. [Online]. Available: https://github.com/pulp-platform/riscv

[14] W. P. Kiat, K. M. Mok, W. K. Lee, H. G. Goh, and I. Andonovic, "A comprehensive analysis on data hazard for RISC32 5-stage pipeline processor," in *Proc. 31st Int. Conf. Adv. Inf. Netw. Appl. Workshops (WAINA)*, Mar. 2017, pp. 154–159.

[15] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design RISC-V Edition: The Hardware Software Interface*. San Mateo, CA, USA: Morgan Kaufmann, 2018.

[16] M. Zulkifli, Y. Yudhanto, N. Soetharyo, and T. Adiono, "Reduced stall MIPS architecture using pre-fetching accelerator," in *Proc. Int. Conf. Electr. Eng. Informat.*, Aug. 2009, pp. 611–616.

[17] (2020). *SCR1 Microcontroller Core*. [Online]. Available: http://www.Syntacore.com

[18] Openhwgroup.*Cv32e40p*. Accessed: Jul. 21, 2024. [Online]. Available: https://github.com/openhwgroup/cv32e40p

[19] (2020). *E21-Sifive*. [Online]. Available: http://www.sifive.com

[20] (2020). *Risc-V: N22-Andes Technology*. [Online]. Available: http://www.andestech.com

[21] C. Wolf. (2020). *Picorv32*. [Online]. Available: https://github.com/cliffordwolf/picorv32

[22] P. Bernardi et al., "On the functional test of the register forwarding and pipeline interlocking unit in pipelined processors," in *Proc. 14th Int. Workshop Microprocessor Test Verification*, Dec. 2013, pp. 52–57.

[23] J. Lu, X. Zhou, and J. Wang, "A novel dynamic scheduling algorithm of data hazard for embedded processor," in *Proc. 7th Int. Conf. ASIC*, Oct. 2007, pp. 28–31.

[24] M.-A. Yong, W.-K. Lee, K.-M. Mok, D. C.-K. Wong, and S.-K. Teoh, "Control and data hazard resolving schemes for asynchronous RISC32 5-stage pipeline processor," in *Proc. IEEE Ind. Electron. Appl. Conf. (IEACon)*, Oct. 2022, pp. 196–201.

[25] H. Yongjie, Z. Chenxi, and L. Jiangfeng, "Design and implementation of the ROB algorithm simulator," in *Proc. IEEE Int. Conf. Data Sci. Comput. Appl. (ICDSCA)*, Oct. 2021, pp. 1–5.

[26] C.-H. Cheng, "Design example of useful memory latency for developing a hazard preventive pipeline high-performance embedded-microprocessor," *Hindawi Publishing Corp.*, vol. 2013, no. 1, pp. 1–10, 2013.

[27] Y. He, H. Wan, B. Jiang, and X. Gao, "A method to detect hazards in pipeline processor," in *Proc. MATEC Web Conf.*, vol. 139, 2017, p. 00085.

[28] D. Su, Y. Li, and B. Yuan, "SSR: A stall scheme reducing bubbles load-use hazard RISC-V pipeline," *IEEE Access*, vol. 10, pp. 54967–54977, 2022.

[29] *Fibonacci-Benchmark*. [Online]. Available: https://zwitterion.org/software/fibonacci-benchmark/fibonacci-benchmark.html

[30] P. Tsigas and Y. Zhang, "A simple, fast parallel implementation of quicksort and its performance evaluation on SUN enterprise 10000," in *Proc. 11th Euromicro Conf. Parallel, Distrib. Netw.-Based Process.*, Jan. 2003, pp. 372–381. Accessed: Jul. 21, 2024.

[31] S. Shukla, Utkarsh, M. Azam, and K. C. Ray, "An efficient fault-tolerant instruction decoder for RISC-V based dual-core soft-processors," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 12, pp. 4816–4825, Dec. 2023.

[32] D. Meidanis, K. Georgopoulos, and I. Papaefstathiou, "FPGA power consumption measurements and estimations under different implementation parameters," in *Proc. Int. Conf. Field-Program. Technol.*, Dec. 2011, pp. 1–6.

[33] S. Kumar and K. C. Ray, "An energy-efficient adaptive two stage pipeline RISC-V based soft-processor for edge-computing applications," *Int. J. Parallel, Emergent Distrib. Syst.*, pp. 1–15, Dec. 2024.

[34] A. Sreekumar, B. S. Shankar, and B. N. K. Reddy, "Integrating error correction and detection techniques in RISC-V processor microarchitecture for enhanced reliability," *Integration*, vol. 100, Jan. 2025, Art. no. 102282.

[35] *Rv32I-Tangnano9k*. Accessed: Jul. 21, 2024. [Online]. Available: https://github.com/yutyan0119/rv32I-TangNano9K

[36] I. Elsadek and E. Y. Tawfik, "RISC-V resource-constrained cores: A survey and energy comparison," in *Proc. 19th IEEE Int. New Circuits Syst. Conf. (NEWCAS)*, Jun. 2021, pp. 1–5.

[37] Y.-H. Cheng, L.-B. Huang, Y.-J. Cui, S. Ma, Y.-W. Wang, and B.-C. Sui, "RV16: An ultra-low-cost embedded RISC-V processor core," *J. Comput. Sci. Technol.*, vol. 37, no. 6, pp. 1307–1319, Dec. 2022.

[38] C. Wei Peng et al., "Performance evaluation of RISC-V microcontroller system on FPGA: A study of the NEORV32 core," *Int. J. Integr. Eng.*, vol. 16, no. 1, pp. 312–317, Apr. 2024. [Online]. Available: https://publisher.uthm.edu.my/ojs/index.php/ijie/article/view/16362

[39] T. Zheng, G. Cai, and Z. Huang, "A soft RISC-V processor IP with high-performance and low-resource consumption for FPGA," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2022, pp. 2538–2541.

[40] S. Shukla, P. K. Jha, and K. C. Ray, "An energy-efficient single-cycle RV32I microprocessor for edge computing applications," *Integration*, vol. 88, pp. 233–240, Jan. 2023.



Sujeet Kumar received the B.E. and M.Tech. degrees in 2010 and 2013, respectively. He is currently pursuing the Ph.D. degree in electrical engineering with the Indian Institute of Technology Patna, India. Before joining his current position, he was an Assistant Professor with the Department of Electronics and Communication Engineering, IMS Engineering College, Ghaziabad, India. His research interests include RISC-V-based pipeline processor design, lightweight edge computing system design, digital VLSI design, FPGA-based system design, ASIC implementations, and fault-tolerant design.



Kailash Chandra Ray (Member, IEEE) received the B.E. degree in electrical engineering from Orissa Engineering College, Bhubaneswar, India, in 1997, the M.Tech. degree in electrical engineering from NIT, Jamshedpur, India, in 2000, and the Ph.D. degree in electronics and electrical communication engineering from IIT Kharagpur, India, in 2009. He was a Technical Member Staff (a Design Engineer) with CMOS Chips, Bengaluru, India, from 2001 to 2003. He was also a Lecturer with IIIT Allahabad, India, from 2008 to 2010. He was an Assistant Professor with the Department of Electrical Engineering, Indian Institute of Technology Patna, India, from 2010 to 2018, where he has been an Associate Professor since 2018. His research interests include very large-scale integration (VLSI) architectural design, VLSI signal processing, FPGA-based system design, and RISC-V-based lightweight edge computing systems.

Authorized licensed use limited to: UNIVERSITAS GADJAH MADA. Downloaded on August 24,2026 at 02:50:22 UTC from IEEE Xplore. Restrictions apply.